1 Workflow definition

Workflows are instances of the *Workflow* class, and are defined in specially developed domain specific language. According to the metamodel Figure 1, a workflow can be a Task, specified by an ID; a Sequence of two or more subworkflows, to be performed in order; a Loop comprising a body (a (sub)workflow) to be executed continuously whilst its guard (boolean expression) holds; or finally, a Decision comprising one or more options, each of which is a GuardedWorkflow. The guards (boolean expressions) of all options are evaluated at runtime. When one guard holds, its associated body is executed. If no guard holds, the execution of the Decision terminates with no task executed; equivalent to the predefined task with ID Skip. If multiple guards hold, the agent must select the appropriate body to execute.

We reserve task ID "Skip" to indicate that no action is required and that the system should move to the next process in the workflow.

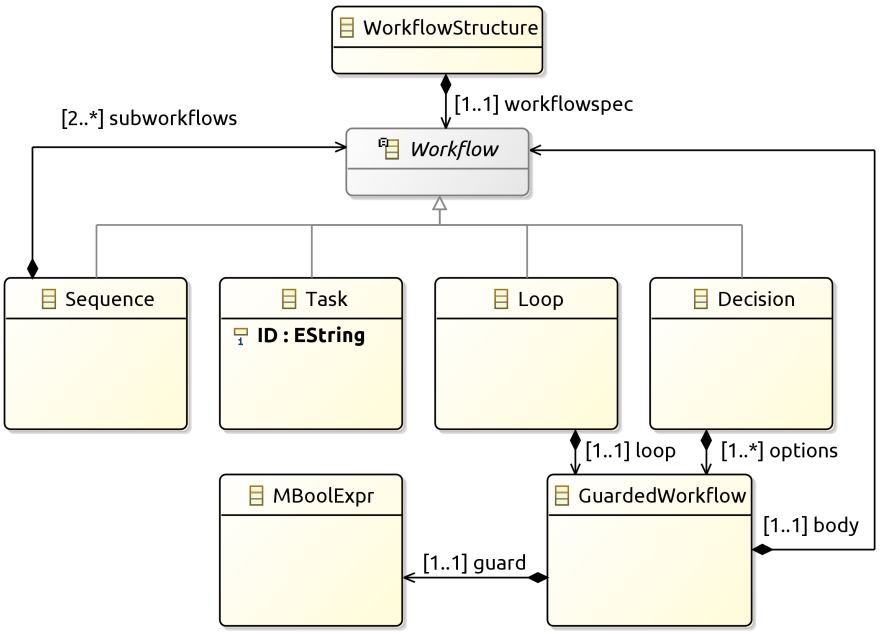

Fig. 1. Workflow metamodel

2 Wellformedness

Workflow w, an instance of the Workflow (Figure 1), and a set of $n \geq 1$ conflict and redundancy-free SLEEC rules must be formed so:

Assumption 1) all rule **triggerEvent**s are of form taskIdStart or taskIdEnd, where taskId represents the ID of a Task that may be present in w, and the suffix indicates whether the trigger event refers to the start or the end of the Task;

Assumption 2) all rule **responseEvent**s and **defeaterResponse**s correspond to actual capabilities of the agent and are not associated with any task from w,

3 Analysis of workflow adaptation algorithm

We provide formal results that establish the correctness and complexity of our workflow adaptation. The first of these shows that our adaptation algorithm does not impact the functionality of the workflow.

Theorem 1. *Given a workflow with the structure from Figure 1, and a SLEEC ruleset satisfying Assumptions 1 and 2 from Section 1, the function* AdaptWorkflow *from Algorithm 1 returns a workflow that is functionally equivalent to the original workflow, provided that any tasks added by* AdaptWorkflow *are disregarded (or, equivalently, replaced by the predefined task Skip).*

Proof. The proof is by induction over the *workflow* structure.

Base case: Let *workflow* be a Task. AdaptWorkflow either returns this task unchanged (Algorithm 1, line 7)—if no SLEEC rules refer to this task, or returns a Sequence to which the task is added in line 14, and whose other tasks are all derived from SLEEC rules. In both scenarios, the resulting workflow is equivalent to the original task (in the sense specified in the theorem).

Inductive hypothesis: Assume that the theorem holds for: (i) the sub-workflows of the Sequence from lines 20–26; (ii) the body of every option of the Decision from lines 27–32; and (iii) the body of the Loop from lines 33–35.

Inductive step: (i) Let *workflow* be a Sequence of sub-workflows. AdaptWorkflow returns a new *sequence* (line 26) that combines the SLEEC-ADAPT*ed* variants of these sub-workflows, preserving their original order. Since the theorem holds for the SLEEC-ADAPT*ed* sub-workflows (by inductive hypothesis), it will also hold for their combination within the returned *sequence*. (ii) Let *workflow* be a Decision. AdaptWorkflow returns a new Decision with the same number of options, each with the same guard as an original option, and the SLEEC-ADAPT*ed* variant of that original option's body. As the theorem holds for these SLEEC-ADAPT*ed* option bodies (by inductive hypothesis), and the guards are unchanged in the returned *sequence*, the theorem will also hold for this *sequence*. (iii) An argument similar to that used for a Decision in (ii) shows that the theorem also holds when *workflow* is a Loop.

Having covered all cases from the switch statement starting in line 2 of AdaptWorkflow (Task in the base case, and Sequence, Decision and Loop in the inductive step), we conclude that the theorem holds for any *workflow*.

The next theorem demonstrates that the adapted workflow produced by our algorithm is SLEEC compliant.

Algorithm 1 SLEEC-ADAPT Workflow Adaptation

```
function AdaptWorkflow(workflow, ruleset)
  switch workflow.type do
    case Task:
      startRules ← GetRulesByTrigger(workflow.id + "Start")
      endRules ← GetRulesByTrigger(workflow.id + "End")
      if startRules = ∅ ∧ endRules = ∅ then
        return workflow
      end if
      sequence ← new Sequence
      for each rule ∈ startRules do
        SLEECwf ← GenTasksFromSLEEC(rule)
        sequence ← AddSubworkflow(sequence, SLEECwf)
      end for
      sequence ← AddSubworkflow(sequence, workflow)
      for each rule ∈ endRules do
        SLEECwf ← GenTasksFromSLEEC(rule)
        sequence ← AddSubworkflow(sequence, SLEECwf)
      end for
      return sequence
    case Sequence:
      sequence ← new Sequence
      for each subwf ∈ workflow.subworkflows do
        SLEECwf ←AdaptWorkflow(subwf, ruleset)
        sequence ←AddSubworkflow(sequence, SLEECwf)
      end for
      return sequence
    case Decision:
      SLEECoptions ← []
      for each option ∈ workflow.options do
        SLEECoptions ← SLEECoptions([new GuardedWorkflow(option.guard,
                        AdaptWorkflow(option.body, ruleset)]
      end for
      return new Decision(SLEECoptions)
    case Loop:
      SLEECbody ← AdaptWorkflow(workflow.loop.body, ruleset)
      return new Loop(new GuardedWorkflow(workflow.loop.guard, SLEECbody))
end function

function GenTasksFromSLEEC(rule)
  responseWf ← new Task(rule.response)
  if rule.triggerGuard ≠ nil then
    responseWf ← new Decision(
          [new GuardedWorkflow(rule.triggerGuard,responseWf)])
  end if
  return AddDefeaters(responseWf, rule.defeaters)
end function

function AddDefeaters(responseWf, dfs)
  if dfs = nil then
    return responseWf
  else
    return new Decision([ new GuardedWorkflow(last(dfs).guard,
                          new Task(last(dfs).response)),
                          new GuardedWorkflow(¬last(dfs).guard,
                          AddDefeaters(responseWf,front(dfs)))])
  end if
end function
```

Theorem 2. *Under the same premises as those specified for Theorem 1,* AdaptWorkflow *returns a workflow that satisfies all SLEEC rules contained within the ruleset provided as the second argument for the function.*

Proof. AdaptWorkflow performs a depth-first traversal of the input *workflow*, processing each task exactly once (lines 3–19). For every task, it determines the SLEEC rulesets associated with the task's start and end in lines 4 and 5, respectively. SLEEC compliance is achieved by adding tasks derived from each *rule* within these rulesets via the function GenTasksFromSLEEC, both before the original task (lines 10–13) and after the original task (lines 15–18). We show that this function operates correctly by induction over the number of defeaters d in the *rule*.

Base case: When the *rule* handled by GenTasksFromSLEEC contains no defeaters ($d = 0$), a task corresponding to the *rule*'s response is created in line 38. This task is then placed into the body of a Decision if the *rule* specifies a triggerGuard (lines 39–41). The resulting sub-workflow—which ensures compliance with the SLEEC *rule*—is returned without further modification, as the call to AddDefeaters (line 42) has no effect in the absence of defeaters.

Inductive hypothesis: Assume that for any SLEEC *rule* with fewer than N defeaters ($d < N$), the sub-workflow returned by GenTasksFromSLEEC ensures compliance with that *rule*.

Inductive step: Consider a *rule* with $d = N$ defeaters, where $N > 0$. The resulting sub-workflow is returned following the call to AddDefeaters from line 42. This function handles the sequence of $N > 0$ defeaters recursively (line 48) by:

1. Making a recursive call to itself with a sequence only containing the first $N - 1$ defeaters. By the inductive hypothesis, this call returns a sub-worklow that satisfies the SLEEC rule with respect to those $N - 1$ defeaters.
2. The N-th defeater is then handled by constructing a two-option Decision (line 48). The first of these options performs the task derived from the N-th defeater's response (if that defeater's guard holds), while the second option executes the correct sub-workflow associated with the first $N - 1$ defeaters. This precisely matches the intended semantics of the SLEEC *rule*, ensuring that the sub-workflow returned by GenTasksFromSLEEC complies with the entirety of the *rule*, as required by the theorem.

The final result from this section establishes the time complexity of our workflow adaptation algorithm.

Theorem 3. *The function* AdaptWorkflow *executes in at most* $\mathcal{O}(tr(d+1))$ *steps, where: t is the number of tasks in the input workflow; r is the number of rules in the input ruleset; and d is the maximum number of defeaters associated with any single SLEEC rule within the ruleset.*

Proof. GetRulesByTrigger (lines 4 and 5) amounts to a 'for' loop inspecting and appending rules to a set. Thus it has time complexity $\Theta(r)$. The auxiliary function AddSubWorkflow appends a sub-workflow to a Sequence by setting the pointer of the last element from the original Sequence to the first element of the appended sub-workflow. This is a constant-time operation.

AdaptWorkflow is divided into four cases. When the *workflow* is a Task (the base case, line 3), two calls are made to GetRulesByTrigger ($\Theta(r)$) to

compile relevant SLEEC rulesets. A loop through the first ruleset (*startRules*) follows (lines 10–13) and invokes GenTasksFromSLEEC and AddSubWorkflow to derive sub-workflows from each of the up to r rules from *startRules*. These have time complexities of $\mathcal{O}(d+1)$ (since GenTasksFromSLEEC calls AddDefeaters once, and this function then invokes itself recursively at most of d times, once for each defeater from the *rule* handled by GenTasksFromSLEEC), and $\Theta(1)$, respectively. This gives $\mathcal{O}(r(d+1))$ complexity for this loop. Another loop of analogous structure and time complexity follows (lines 15–18). Thus, the complexity to handle the *workflow*'s t tasks is $\mathcal{O}(tr(d+1))$.

For other cases, AdaptWorkflow iterates through the *workflow*'s sub-workflows. For Decisions, Loops and every task of Sequences, a constant-time operation is performed each iteration to add the adapted sub-workflow (excluding the time to adapt individual tasks, whose complexity we already considered for the base case). Thus, the non-base cases of AdaptWorkflow add a time complexity of $\Theta(w)$, where w is the number of workflow elements.

Combining the complexities derived so far results in an overall time complexity of $\Theta(w) + \mathcal{O}(tr(d+1))$, which simplifies to $\mathcal{O}(tr(d+1))$ as the effect of w is negligible.